

NB : La documentation suivante est plus facile à lire et à comparer au code si ce code est copié dans un éditeur de code permettant de visualiser chaque méthode individuellement, plutôt que de le visualiser comme ci-dessus, soit une suite de plus de 200 lignes de codes, avec des retours de ligne forcés par la largeur de la page et des séparations du code causées par la longueur limitée des pages.

Importation

On importe la bibliothèque *java.lang* car ce programme utilise plusieurs méthodes de lecture et de conversion de caractères, d'entiers et de *String* et car la classe *Sudoku* étend la classe *Exception* afin de créer des exceptions spéciales pour ce programme. On importe la bibliothèque *java.io* car ce code utilise les *readers* et *writers* de fichier et d'entrée et de sortie standards. On importe la bibliothèque *java.util* à cause de l'utilisation de la méthode *Array.fill()* dans la méthode *findGrid()*.

Sudoku

La classe *Sudoku* est déclarée *public* cette classe pourrait être étendue ou utilisée afin de créer un solutionneur de sudoku ou d'autres applications où il est nécessaire d'avoir accès au maximum de propriétés de la classe mère possibles.

Exceptions

Les exceptions *invalidGrid*, *invalidInsertion* et *invalidCommand*, créées par extension de la classe *Exception*, servent à lancer et attraper des exceptions propres au programme, par exemple une grille de sudoku invalide, et à afficher le message correspondant à la situation rencontrée dans le programme.

findGrid()

La méthode *findGrid()* est déclarée *public* car elle est utilisée afin d'aller trouver la grille de sudoku dans un fichier donné en argument et la retourner en tableau à deux dimension représentant la grille de manière plus facile à manipuler pour le programme. Il pourrait donc être nécessaire de l'appeler si par exemple, on souhaitait étendre la classe *Sudoku*. Cette même idée explique le qualificatif *public* donné aux autres méthodes le possédant. La méthode *findGrid()* utilise les lecteurs de fichier afin d'enregistrer la ligne contenant les informations de la grille dans une variable, puis lit cette ligne en prenant le format donné dans l'énoncé du devoir comme format normal (c'est-à-dire le numéro de ligne(x) suivi du numéro de colonne(y) suivi de la valeur dans la case(z) suivis d'un espace, sachant que la méthode *affichage()* transposera cette matrice à deux dimensions). Si la ligne finit d'être lue ou si elle était vide (ce qui lance les exceptions *NullPointerException* et *StringIndexOutOfBoundsException* attrapées par le bloc *try-catch*), on a fini d'enregistrer la grille à partir de la ligne, donc *done* est passé à *true*. Si les valeurs trouvées aux positions où devraient être les valeurs de x, y et z ne sont pas des valeurs de sudoku légales, c'est-à-dire que x et y ne sont pas entre 0 et 8 inclusivement et z n'est pas entre 1 et 9 inclusivement, le fichier donné en argument ne contient pas une grille de sudoku de format normal. Donc, on arrête d'essayer de l'enregistrer dans la matrice en passant *done* à *true*, et on indique que cette grille est illégale en passant *legalgrid* à *false*. Si *legalgrid* est passé à *false*, on affiche dans la console que ce fichier ne contient pas une bonne grille de sudoku et on réinitialise la matrice à retourner par la méthode à une matrice à deux dimensions nulles. Finalement, si une exception *IOException* est lancée par une quelconque

ligne de code, cela doit être dû à la lecture à partir l'argument *dir*, et cette exception est gérée dans la méthode *main()* (voir plus bas).

checkPresence()

La méthode *checkPresence()* est qualifiée de *protected* car elle n'est pas censée être utilisée à l'extérieur des méthodes de cette classe ou de ses potentielles sous-classes. Elle sert à vérifier si une valeur *k* se trouve déjà dans la ligne *i* ou la colonne *j* (sauf dans la case (*i*, *j*)) dans la grille de sudoku *grid* selon les contraintes de sudoku données par l'énoncé du devoir. On commence en prenant pour acquis que la valeur n'est pas présente, d'où *presence* initialisée à *false*, puis on passe à travers toutes les valeurs de la colonne *j*, et, si une d'entre elle est égal à *k*, on passe *presence* à *true* puis, on sort de la vérification. Le même processus s'effectue ensuite avec la ligne *i* si nécessaire. La valeur finale de *presence* est retournée, c'est-à-dire *true* si la valeur *k* se trouve dans la ligne *i* ou la colonne *j*, et *false* si ce n'est pas le cas.

checkGrid()

La méthode *checkGrid()* est *public* pour les raisons expliquées dans la méthode *findGrid()*, et elle sert à vérifier si les valeurs déjà présentes dans la grille passée en argument respectent les contraintes de sudoku de l'énoncé du devoir. Pour ce faire, elle utilise une variable *notValidGrid* qui est *true* si la grille n'est pas valide et *false* si la grille l'est. On prend pour acquis que la grille est valide. Donc la variable est initialisée à *false*. Puis, on observe case à case la grille passée en argument. Si la case n'est pas « vide », on utilise la méthode *checkPresence()* afin de voir si la valeur s'y trouvant respecte les contraintes, et dès que cette fonction retourne *true*, *checkGrid()* lance l'exception personnalisée *invalidGrid* afin d'alerter l'utilisateur que sa grille n'est pas valide. Finalement, la fonction retourne la valeur inverse de *notValidGrid* (*true* si la grille est valide et *false* sinon) même s'il est uniquement possible de se retrouver à cette étape si aucune exception n'a été lancée. Ceci implique alors que si une valeur est retournée, elle sera toujours *true*.

insertVal()

La méthode *insertVal()* est *public* pour les raisons expliquées dans la méthode *findGrid()*, et elle sert à insérer une valeur *val* dans la grille *grid* à la position *posx*, *posy*. Elle retourne *true* si l'insertion est réussie et lance l'exception *invalidInsertion* et retourne *false* si l'insertion est impossible. On commence par vérifier que la valeur à insérer est entre 1 et 9 inclusivement (les valeurs légales de Sudoku). On vérifie aussi que les positions où on souhaite insérer la valeur sont entre 0 et 8 inclusivement (les bornes de la matrice utilisée pour stocker la grille). Ensuite, on utilise *checkPresence()* pour savoir si *val* se trouve déjà dans la ligne *posx* ou la colonne *posy*. Si aucun de ces tests déclenche l'exception *invalidInsertion* et passe *insertion* à *false*, *insertion* est passée à *true* et l'insertion est faite, puis la valeur de *insertion* est retournée.

affichage()

La méthode *affichage()* est *public* pour les raisons expliquées dans la méthode *findGrid()*, elle sert à afficher la transposée de la matrice manipulée par le programme. Ceci s'effectue en deux étapes. On crée un tableau et y enregistre la

transposée de la grille passée en argument une case à la fois. Ensuite, on l'affiche dans la console en utilisant un formatage de texte facilitant la lecture. (Preuve avec grilles asymétriques en annexe.)

saveChanges()

La méthode *saveChanges()* est *public* pour les raisons expliquées dans la méthode *findGrid()*. Elle sert à enregistrer la grille manipulée par le programme dans un fichier texte appelé *newfile.txt* selon le format normal. On crée le fichier dans le répertoire passé en argument et on utilise un *writer* de bas niveau afin d'ajouter, un caractère à la fois, les valeurs non-nulles de la grille de sudoku matricielle passée en argument selon le format normal. Afin de savoir quelles valeurs écrire dans *newfile.txt*, on observe la grille matricielle case à case et, si la case n'est pas « vide », on ajoute au document texte la ligne, la colonne, la valeur à cette position et un espace, à la suite de ce qui se trouve dans le fichier, tel que spécifié par le format normal.

Sudoku()

La méthode *Sudoku()* est un constructeur permettant la création de nouveaux objets contenant les méthodes déclarées ci-haut afin de pouvoir les utiliser dans la méthode *main()*, qui est *static*, sans avoir à les déclarer *static* pour aucune autre raison.

throwIC()

Cette méthode sert uniquement à être appelée lorsqu'une exception *invalidCommand* doit être lancée de façon encapsulée dans des blocs *try-catch* de la méthode *main()*. Elle est déclarée *public* pour les raisons expliquées dans la méthode *findGrid()*.

main()

Cette méthode est, comme toujours, qualifiée de *public* et *static*. La création d'un nouvel objet *Sudoku* (ligne 144) sert uniquement à permettre l'appel des méthodes décrites précédemment sans avoir à déclarer ces méthodes *static* pour aucune autre raison que leur usage dans *main()*. Tout le code de cette méthode est contenu dans un bloc *try-catch* afin d'attraper les diverses exceptions lancées par les méthodes appelées et de les gérer en affichant sur la console le message correspondant.

L'idée derrière l'organisation de cette méthode est que, une fois le programme lancé et l'utilisateur ayant joué sur une grille de sudoku de son choix, il est possible que cet utilisateur souhaite continuer à jouer sur la même grille, qu'il souhaite jouer sur une nouvelle grille, ou qu'il souhaite quitter le jeu. On a donc décidé d'utiliser les commandes *continue* étiquetées afin de minimiser les redondances de codes. De ce fait, après qu'il ait joué, on demande à l'utilisateur ce qu'il souhaite faire et on le renvoie à la boucle *do-while* correspondante à l'étape du processus qu'il souhaite répéter (ouvrir un nouveau fichier ou jouer sur la même grille) ou on termine la méthode *main()* en appelant *return*. C'est pour cette raison que la méthode contient trois boucles *do-while* imbriquées : la première demandant d'entrer le fichier sur lequel jouer et enregistrant cette grille dans la variable *grid1* grâce à *findGrid()*, la deuxième demandant d'entrer la commande de jeu afin d'effectuer cette action sur la grille stockée dans la variable *grid1*, et la dernière demandant à l'utilisateur s'il

souhaite poursuivre sur cette grille, s'il souhaite jouer sur une autre grille ou s'il souhaite quitter le programme, et le renvoyant à l'étape sélectionnée.

La première boucle fait exactement ce qui est décrit ci-haut. Elle a la possibilité de lancer l'exception *IOException*, qui est gérée par le bloc *try-catch* l'encapsulant, si le fichier entré n'est pas un fichier de grille de sudoku de format normal ou si l'entrée de l'utilisateur ne pointe pas sur un fichier.

La deuxième boucle demande à l'utilisateur ce qu'il souhaite faire avec sa grille dans *grid1*. Il a d'abord la possibilité de placer une valeur sur la grille. Le programme lui demande successivement d'entrer la ligne, la colonne et la valeur à insérer dans la grille, puis utilise la méthode *insertVal()* afin de faire l'insertion. Si cette méthode ne lance pas l'exception *invalidInsertion*, on affiche sur la console que l'insertion est réussie. Il est important de noter que les valeurs demandées à l'utilisateur se basent sur ce que l'utilisateur verrait s'il affichait la grille selon la méthode *affichage()*. Si *invalidInsertion* est lancée, elle est gérée par le bloc *try-catch* encapsulant toutes les boucles en affichant le message lui correspondant. La deuxième possibilité d'action de l'utilisateur sur la grille dans *grid1* est de vérifier si la grille est une grille de sudoku valide. On utilise un petit bloc *try-catch* qui utilise la méthode *checkGrid()* qui peut lancer l'exception *invalidGrid*. Si l'exception n'est pas lancée, on affiche à la console que la grille est valide, si elle est lancée, on affiche à la console le message opposé. Ceci permet à l'utilisateur de corriger sa grille si ceci est accidentel car cette erreur n'arrête pas le programme puisqu'elle n'est pas gérée par le bloc *try-catch* encapsulant tout le code de la méthode *main()*. La troisième option donnée à l'utilisateur est d'enregistrer sa grille. On lui demande alors d'entrer le répertoire où il souhaite l'enregistrer et on utilise la méthode *saveChanges()*. Si ces étapes ne lancent pas d'exception *IOException*, on affiche à la console que le fichier *newfile.txt* a bien été enregistré dans le répertoire indiqué. Si une exception *IOException* est lancée, elle est gérée par le bloc *try-catch* encapsulant toutes les boucles *do-while*, affichant ainsi son message personnalisé. Cette exception est causée par une entrée de répertoire non valide, comme l'indique son message. Finalement, la dernière option que l'utilisateur a, est d'afficher la grille stockée dans *grid1* sur la console, simplement par l'appel de la méthode *affichage()*. Si la commande de « jeu » entrée par l'utilisateur ne correspond pas aux options lui étant données, on appelle la méthode *throwIC()* afin d'indiquer que la commande entrée n'est pas valide.

La raison pour laquelle on appelle une fonction qui ne fait que lancer une exception au lieu de lancer directement cette exception, est que le bloc *try-catch* ne semble pas capable d'attraper les exceptions lancées directement à partir de sa section *try*. Il semble uniquement capable d'attraper les exceptions lancées par une fonction ou une méthode exécutée dans sa section *try* comme s'il nécessitait un niveau de détachement afin de pouvoir attraper les exceptions. Puisqu'on souhaite qu'il attrape cette exception, on la lance à partir d'une méthode de la classe dédiée à cette fin.

La dernière boucle *do-while* demande à l'utilisateur s'il souhaite continuer sur la grille qu'il a déjà chargée, s'il souhaite jouer sur une nouvelle grille ou s'il souhaite arrêter de jouer. S'il souhaite continuer, on le renvoie à l'étiquette pointant sur la boucle *do-while* correspondante, et s'il souhaite arrêter, on ferme les lecteurs utilisés pour la lecture du fichier ouvert dans la première boucle *do-while* (les lecteurs utilisés dans la seconde boucle n'étant pas toujours tous ouverts rendu à cette étape, il n'est pas toujours nécessaire de tous les fermer donc ils sont ignorés)

et on utilise *return* afin d'arrêter la méthode *main()*. Si l'utilisateur entre une commande différente de celles spécifiées lors de l'interrogation au début de la boucle, on appelle la méthode *throwIC()* pour la même raison qu'expliquée au paragraphe précédent.

Ces trois boucles ont des conditions de fermetures correspondantes aux commandes possibles de la dernière boucle. La dernière boucle se ferme si la commande entrée correspond à une des trois possibilités établies, la deuxième se ferme lorsque la commande choisie est autre que celle où l'utilisateur continue à jouer sur la même grille, la première se ferme lorsque la commande entrée est autre que celles où l'utilisateur continue à jouer.

Finalement, les exceptions sont gérées en affichant leurs messages spécialisés indiquant sur la console quel problème fut rencontré. On a ajouté ici un *catch* spécial pour l'exception *FileNotFoundException* qui est une extension de la classe *IOException* afin de lui donner un message plus spécifique que celui donné lors des autres exceptions de sa classe mère.

ANNEXE

Voici ce que les fichiers textes contiennent :

- partie1.txt : 001 012 023 034 045 056 067 078 089 102 113 124 135 146 157
168 179 181 203 214 225 236 247 258 269 271 282 304 315 326 337 348
359 361 372 383 405 416 427 438 449 451 462 473 484 506 517 528 539
541 552 563 574 585 607 618 629 631 642 653 664 675 686 708 719 721
732 743 754 765 776 787 809 811 822 833 844 855 866 877 888
- partie2.txt : 006 022 047 053 183 201 223 248 252 265 289 302 318 343 359
367 376 405 429 444 482 517 526 535 561 612 621 636 649 673 687 733
813 827 869 875
- partie3.txt : 074 137 203 275 346 521 558 668 701 714 739 852

```
Command Prompt
C:\Users\loubo\OneDrive\Documents\UNI\TELUQ\H20\INF1220\CODE\TN4>java Sudoku
What file do you wish to play on?
partie1.txt
Do you wish to (enter nombre only):
1.Play on the grid
2.Check if the constraints are respected
3.Save your grid
4.Print out your grid
4
-----
|123|456|789|
|234|567|891|
|345|678|912|
-----
|456|789|123|
|567|891|234|
|678|912|345|
-----
|789|123|456|
|891|234|567|
|912|345|678|
-----
Do you wish to continue?
Enter 1 to continue on this grid, 2 to change up another grid, 3 to stop
2
What file do you wish to play on?
partie2.txt
Do you wish to (enter nombre only):
1.Play on the grid
2.Check if the constraints are respected
3.Save your grid
4.Print out your grid
4
-----
```

```

-----
Do you wish to continue?
Enter 1 to continue on this grid, 2 to charge up another grid, 3 to stop
2
What file do you wish to play on?
partie2.txt
Do you wish to (enter nombre only):
1.Play on the grid
2.Check if the constraints are respected
3.Save your grid
4.Print out your grid
4
-----
|602|073|000|
|000|000|003|
|103|082|509|
-----
|280|039|760|
|509|040|002|
|076|500|100|
-----
|021|690|037|
|000|300|000|
|037|000|950|
-----
Do you wish to continue?
Enter 1 to continue on this grid, 2 to charge up another grid, 3 to stop
2
What file do you wish to play on?
partie3.txt
Do you wish to (enter nombre only):
1.Play on the grid
2.Check if the constraints are respected
3.Save your grid
4.Print out your grid
4
-----
2
What file do you wish to play on?
partie3.txt
Do you wish to (enter nombre only):
1.Play on the grid
2.Check if the constraints are respected
3.Save your grid
4.Print out your grid
4
-----
|000|000|040|
|000|700|000|
|300|000|050|
-----
|000|060|000|
|000|000|000|
|001|008|000|
-----
|000|000|800|
|140|900|000|
|000|002|000|
-----
Do you wish to continue?
Enter 1 to continue on this grid, 2 to charge up another grid, 3 to stop
3
C:\Users\loubo\OneDrive\Documents\UNI\TELUQ\H20\INF1220\CODE\TN4>

```